

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration: 1 Hour
Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

I _M Kavya 192472370_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: kavya M

Annexure A

Scenario Based Assignment

1. AI-Powered Drone for Emergency Medicine Delivery

i. Greedy Best-First Search (GBFS)

Greedy Best-First Search selects the path that appears closest to the destination using only the heuristic function $h(n)$ (straight-line distance). It ignores the cost already spent.

Working:

- Starts from the drone's current location.
- Calculates the heuristic distance to the destination.
- Chooses the node with the smallest heuristic value.
- Repeats until the destination is reached.

Strengths

- Very fast.
- Uses less memory than many search algorithms.
- Suitable for urgent situations where a quick decision is required.

Weaknesses

- Does not guarantee the shortest or safest path.
- May choose blocked roads because it ignores actual travel cost.
- Performs poorly in uncertain or changing environments.
- Can get trapped in dead ends.

ii. A* Search Algorithm

A* evaluates each node using:

$$f(n) = g(n) + h(n)$$

Where:

- $g(n)$ = Actual cost from the start.
- $h(n)$ = Estimated cost to the goal.

Working

1. Start from the source.
2. Compute $f(n)$ for neighboring nodes.

3. Expand the node with the lowest $f(n)$.
4. Update costs when better paths are found.
5. Continue until the destination is reached.

Advantages

- Produces optimal paths when the heuristic is admissible.
- Balances travel cost and estimated distance.
- Avoids unnecessarily expensive routes.
- Handles varying terrain costs better than GBFS.

iii. Impact of Heuristic Accuracy

Accurate Heuristic

- GBFS reaches the destination quickly.
- A* expands fewer nodes and finds the optimal path faster.

Poor Heuristic

- GBFS may choose unsafe or blocked routes.
- A* still finds the optimal path but explores more nodes, increasing computation time.

Thus, heuristic quality directly affects search efficiency.

iv. Effect of Dynamic Changes (Blocked Paths)

When roads suddenly become blocked:

GBFS

- Often needs to restart the search.
- May repeatedly choose blocked paths.

A*

- Updates path costs.
- Can recalculate alternative routes more effectively.
- More reliable in changing environments.

v. Recommended Algorithm

Recommended: A*

Justification

Factor	Greedy BFS	A*
Speed	High	Moderate
Optimality	No	Yes
Handles Variable Cost	No	Yes
Safety	Medium	High
Dynamic Environment	Poor	Better

Conclusion

A* is the most suitable algorithm because it balances speed, optimality, and safety. Although slightly slower than Greedy Best-First Search, it provides safer and more reliable routes for emergency medicine delivery.

2. Smart City Traffic Signal Optimization

Problem Formulation

Objective Function

Minimize:

- Total waiting time
- Fuel consumption
- Traffic congestion

Variables

- Green signal duration
- Red signal duration
- Signal sequence
- Coordination between intersections

Constraints

- Minimum and maximum green time
- Emergency vehicle priority
- Pedestrian crossing time

- Traffic regulations

Why Traditional Search is Inefficient

Traditional search explores every possible signal combination.

For hundreds of intersections:

- Millions of possible states.
- Exponential search space.
- Too slow for real-time traffic management.

i. Hill Climbing

Hill Climbing continuously moves toward a better neighboring solution.

Steps

1. Start with current signal timings.
2. Evaluate traffic performance.
3. Slightly modify timings.
4. Keep changes if performance improves.
5. Repeat until no better solution exists.

Advantages

- Simple.
- Fast.
- Low memory usage.

Limitations

- Gets stuck at local maxima.
- Cannot explore poor intermediate solutions.
- Sensitive to starting point.

ii. Local Maxima, Plateaus, and Ridges

Local Maxima

The system reaches a solution that appears best locally but is not globally optimal.

Example:

Traffic improves at one junction but worsens city-wide.

Plateau

Many neighboring solutions have identical performance.

Example:

Changing signal timing by one second makes no noticeable improvement.

Ridge

Best improvements require multiple coordinated changes.

Example:

Changing only one intersection does not help, but changing several simultaneously reduces congestion.

iii. Simulated Annealing and Genetic Algorithms

Simulated Annealing

- Sometimes accepts worse solutions.
- Escapes local maxima.
- Gradually reduces randomness.
- Eventually converges to a good solution.

Genetic Algorithm

Inspired by natural evolution.

Steps:

1. Generate many signal timing plans.
2. Evaluate fitness.
3. Select the best.
4. Perform crossover.
5. Apply mutation.
6. Repeat.

Advantages:

- Explores many solutions simultaneously.
- Avoids local maxima.
- Suitable for large optimization problems.

iv. Recommended Approach

Recommended: Genetic Algorithm

Justification

- Highly scalable.
- Handles complex constraints.
- Produces near-optimal solutions.
- Adapts to changing traffic patterns.
- Suitable for smart city applications.

3. Autonomous Mars Rover

i. Why Online Search?

Offline search requires the complete map before planning.

Mars rover:

- Unknown terrain.
- Limited sensing.
- Hazards discovered during exploration.

Therefore, online search is required because decisions are made while moving.

ii. Challenges

Partial Observability

- Cannot see distant obstacles.
- Sensor limitations.
- Unknown terrain.

Dynamic Environment

- Dust storms.
- Loose rocks.
- Slippery surfaces.
- Sensor noise.

iii. Internal Model

The rover continuously updates its knowledge.

Model contains:

- Safe paths.
- Obstacles.
- Sample locations.
- Hazard zones.
- Previously visited areas.

Each new observation updates the internal map.

iv. Online vs Other Search

Feature	Uninformed	Informed	Online
Complete Map	Required	Required	Not Required
Uses Heuristic	No	Yes	Optional
Learns While Moving	No	No	Yes
Suitable for Mars	No	Limited	Yes

v. Recommended Approach

Recommended: Online Search Agent

Reasons:

- Handles uncertainty.
- Learns continuously.
- Updates decisions in real time.
- Maximizes safety.
- Suitable for unknown environments.

4. University Exam Timetable (Constraint Satisfaction Problem)

Problem Formulation

Variables

Each course represents one variable.

Example:

- C1
- C2

- C3
- C4

Domains

Possible exam slots.

Example:

- Monday Morning
- Monday Afternoon
- Tuesday Morning
- Tuesday Afternoon

Constraints

1. No overlapping exams for the same student.
2. Limited exam halls.
3. Subject order constraints.
4. Faculty availability.
5. Special accommodations.
6. Hall capacity limits.

i. Variables, Domains, Constraints

Variables:

Courses.

Domains:

Available time slots.

Constraints:

Rules that every timetable must satisfy.

ii. Backtracking Search

Working:

1. Assign a time slot.
2. Check constraints.
3. If valid, continue.

4. If conflict occurs, backtrack.
5. Try another slot.
6. Continue until all courses are assigned.

Advantages:

- Guarantees a valid solution.
- Easy to implement.

iii. Forward Checking

Forward checking removes invalid choices immediately.

Example:

If Course A is scheduled Monday Morning,

then Monday Morning is removed from the domain of conflicting courses.

Benefits:

- Detects conflicts early.
- Reduces search space.
- Faster than plain backtracking.

iv. Real-world Challenges

Challenges:

- Thousands of students.
- Large number of courses.
- Limited halls.
- Faculty constraints.
- Last-minute changes.

Recommended Strategy

Backtracking + Forward Checking + Constraint Propagation.

Benefits:

- Faster.
- Scalable.
- Reduces unnecessary search.

5. Strategic Game AI (Minimax & Alpha-Beta Pruning)

i. Minimax Algorithm

Minimax assumes:

- AI tries to maximize its score.
- Opponent tries to minimize the AI's score.

Process:

1. Generate game tree.
2. Evaluate terminal states.
3. Propagate values upward.
4. Select the move with maximum guaranteed benefit.

ii. Computational Challenges

Large game trees cause:

- Huge branching factor.
- Exponential growth.
- High memory usage.
- Slow decision making.
- Difficult real-time performance.

iii. Alpha-Beta Pruning

Alpha-Beta Pruning removes branches that cannot affect the final decision.

Alpha (α)

Best score AI can guarantee.

Beta (β)

Best score opponent can guarantee.

If

$$\alpha \geq \beta$$

remaining branches are ignored because they cannot improve the outcome.

Benefits:

- Searches fewer nodes.
- Faster execution.
- Produces the same optimal decision as Minimax.
- Suitable for real-time games.

iv. Evaluation Function

Example Evaluation Function:

$$\text{Score} = (\text{Material Advantage} \times 5) + (\text{Map Control} \times 3) + (\text{Resources} \times 2) + (\text{Health} \times 2) - (\text{Enemy Threat} \times 4)$$

Factors:

- Number of units.
- Remaining health.
- Available resources.
- Strategic positions.
- Distance to enemy.
- Defensive strength.
- Attack opportunities.

This function estimates the strength of a game state without exploring the entire game tree.

v. Recommended Strategy

Recommended: Alpha-Beta Pruning with Minimax

Justification

Factor	Minimax	Alpha-Beta
Optimality	Yes	Yes
Speed	Slow	Faster
Memory	High	Lower
Real-Time Decision	Difficult	Suitable
Scalability	Limited	Better

Conclusion:

Alpha-Beta Pruning combined with Minimax is the best choice because it maintains optimal decision-making while significantly reducing computation time. This allows the AI to respond quickly in real-time strategy games while preserving high-quality moves.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand